

The Pretraining Revolution

Impact on NLP Performance

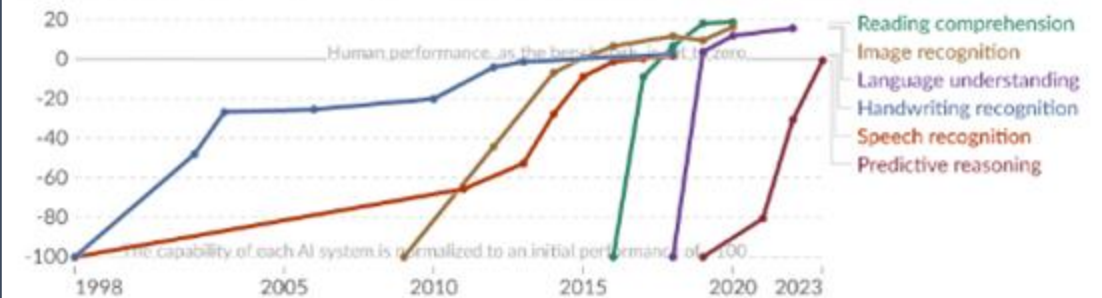
Pretrained language models initiated a profound paradigm shift in natural language processing. By reviewing performance metrics from 2000 to 2020, we observe a dramatic acceleration.

- **Surpassing Human Baselines:** Across diverse datasets including SQuAD (Question Answering) and GLUE (General Language Understanding), models rapidly approached and eventually surpassed the human baseline (0.0) between 2018 and 2020.
- **The Vertical Leap:** Zooming into the 2018-2021 timeline highlights sharp, vertical performance gains.
- **The Catalyst:** This inflection point directly corresponds to the introduction of BERT and its variants, proving that massive pretraining is the key to unlocking deep linguistic understanding.

Test scores of AI systems on various capabilities relative to human performance

Our World
in Data

Within each domain, the initial performance of the AI is set to -100. Human performance is used as a baseline, set to zero. When the AI's performance crosses the zero line, it scored more points than humans.



Data source: Kiela et al. (2023)

CC BY

Note: For each capability, the first year always shows a baseline of -100, even if better performance was recorded later that year.

Scaling Unsupervised Learning

The Foundation Model Pipeline

The modern AI pipeline has shifted away from narrow, task-specific architectures toward massive, generalized "Foundation Models."

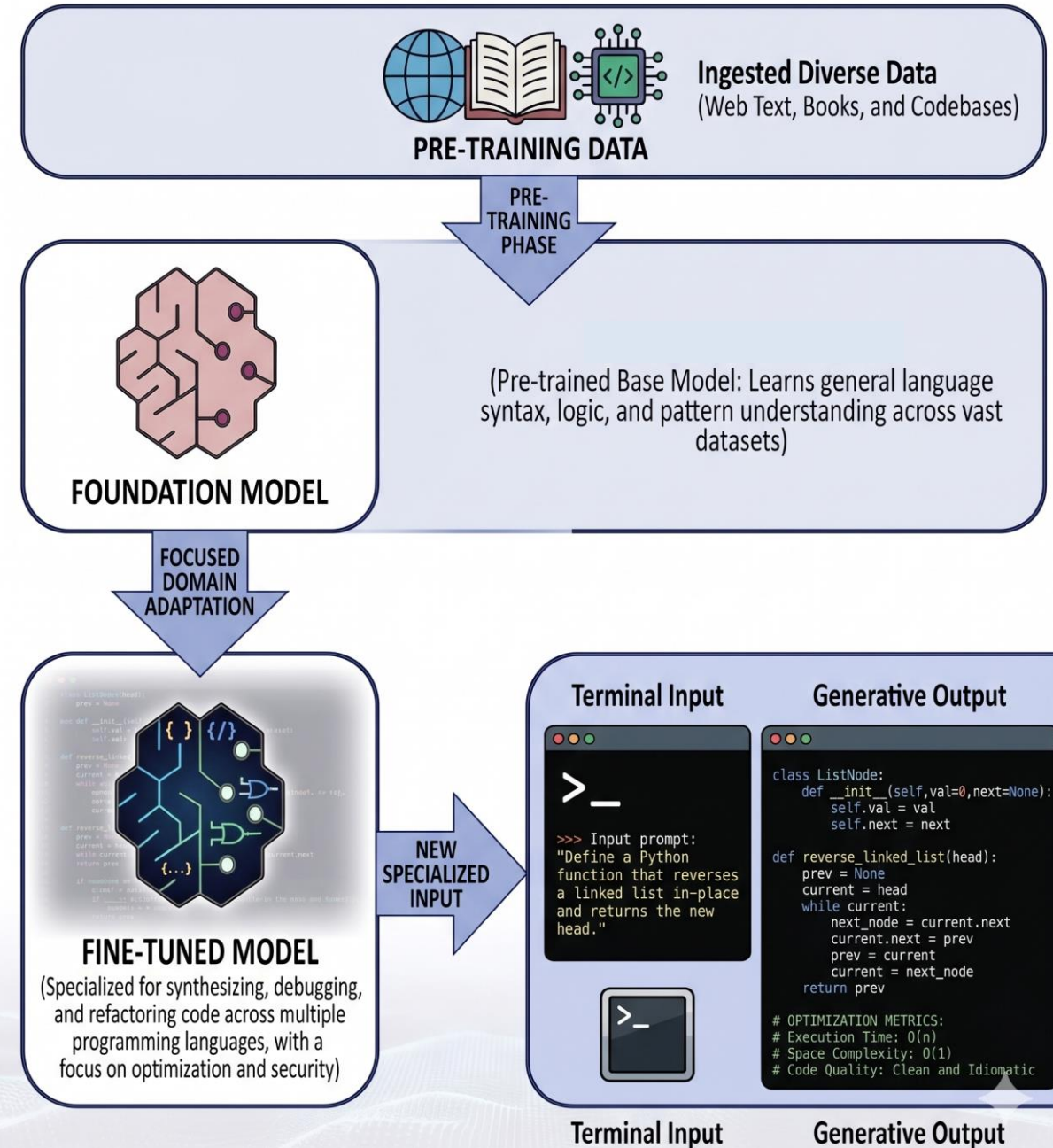
Phase 1: Training

Massive volumes of diverse, unstructured, and unlabeled data (Text, Images, Speech, 3D Signals) are aggregated. Unsupervised learning processes this data to train a central foundation model, ensuring scalability without the bottleneck of human labeling.

Phase 2: Adaptation

Once pretraining concludes, the general knowledge encoded in the foundation model is adapted to handle a wide array of specialized downstream tasks, such as sentiment analysis or object recognition.

LINEAR FOUNDATION MODEL ADAPTATION: THE PATHWAY TO CODE GENERATION



Learning via Reconstruction

What exactly can a model learn simply by reconstructing hidden parts of an input sequence? The answer spans factual knowledge, grammar, and complex reasoning.

Factual Knowledge

"Stanford University is located in **California**."

Semantic Categories

"I went to the ocean to see the fish, turtles, seals, and **whales**."

Grammar & Phrasing

"I put **the** fork down on the table."

Sentiment Inference

"Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was **terrible**."

Coreference Resolution

"The woman walked across the street, checking for traffic over **her** shoulder."

Reading Comprehension

"Iroh went into the kitchen to make some tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the **kitchen**."

Logical Sequences

"I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, **21**,"

Pretraining / Finetuning

This two-step paradigm acts as a highly effective parameter initialization technique, drastically outperforming models trained from scratch.

Step 1: Pretrain

Objective: Language Modeling

The model leverages massive, web-scale amounts of unlabeled text. By playing the "reconstruction game," it builds a dense, mathematical understanding of general language structures, facts, and logic. This phase is computationally expensive but only needs to be performed once, generating a highly capable baseline model.

Step 2: Finetune

Objective: Target Task

The pretrained weights are applied to a specific downstream application (e.g., Sentiment Analysis, Question Answering). Using a much smaller, carefully labeled dataset, the model's general knowledge is adapted to the specific nuances and output formats required by the user prompt.

SGD and Model Finetuning

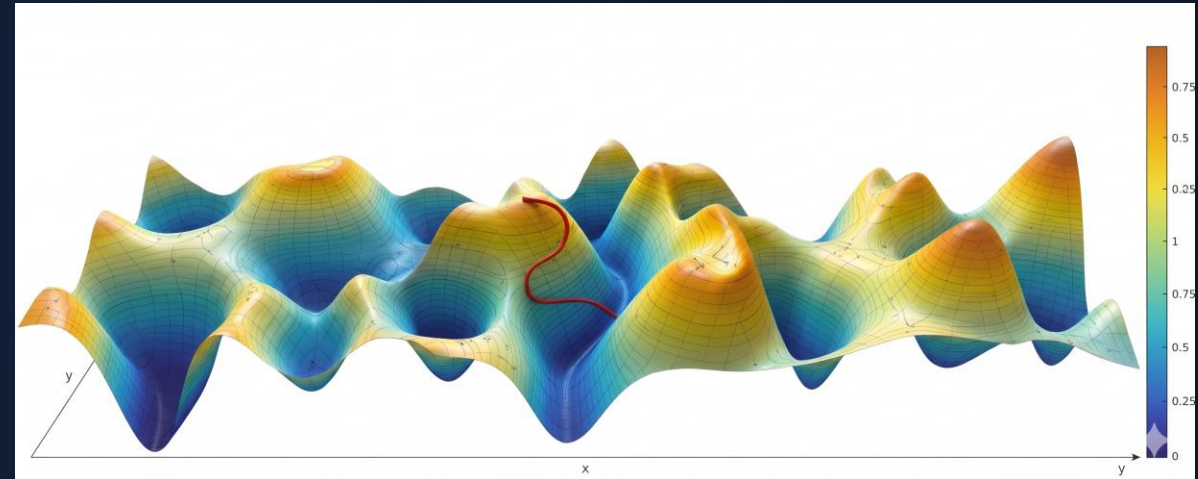
The Mechanics of Parameter Initialization

From a neural network optimization perspective, the pretrain/finetune paradigm creates an ideal loss landscape scenario.

$$\min_{\theta} \mathcal{L}_{\text{finetune}}(\theta)$$

$$\min_{\theta} \mathcal{L}_{\text{pretrain}}(\theta)$$

- **Initial Approximation:** Pretraining navigates the massive parameter space to find a minimum of the pretraining loss, providing excellent initial parameters, denoted as the θ -hat above.
- **Local Generalization:** Finetuning minimizes task-specific loss starting directly from this point. Stochastic Gradient Descent (SGD) tends to stay relatively close to these initialized weights, finding local minima that generalize exceptionally well.



Why **Unsupervised** Learning?

Why shift away from targeted, supervised tasks like Question Answering (QA)? The answer lies entirely in the available scale of data.

Dataset Source	Approximate Token Count	Learning Type
SQuAD 2.0 (Stanford QA)	< 50 Million	Supervised (Labeled)
DCLM-pool	240 Trillion	Unsupervised (Unlabeled)
Indexed Internet Text	~ 510 Trillion	Unsupervised (Unlabeled)

A 10 Million Times

Gap

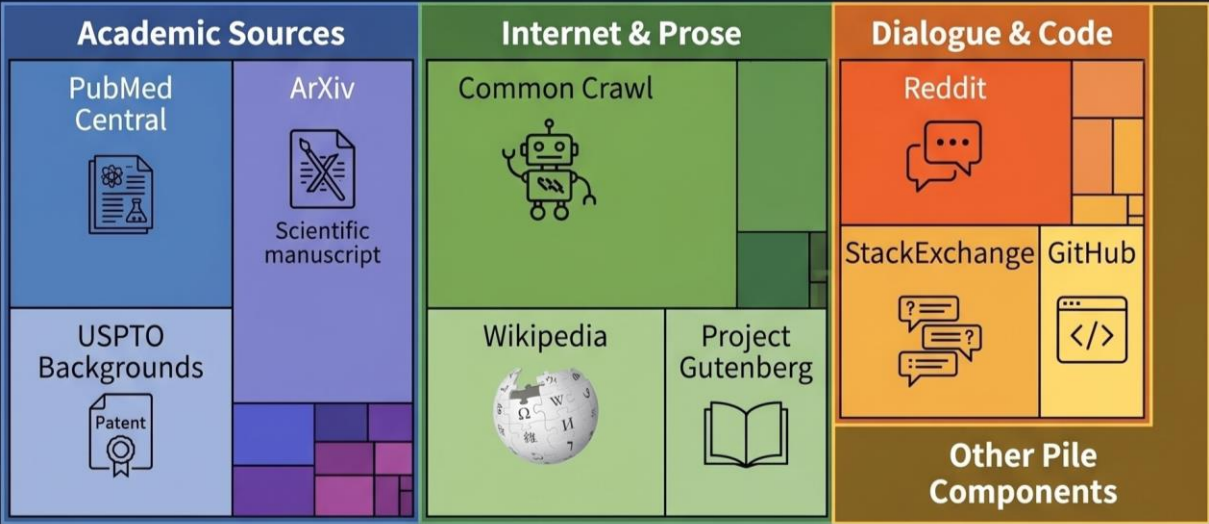
By relying on unsupervised learning over the open internet, models bypass the human labeling bottleneck, unlocking a scale that guarantees rapid capability progression.

Massive Dataset Diversity

Composition of "The Pile"

To train highly capable foundation models, datasets cannot just be large; they must be extraordinarily diverse to cover any potential downstream task.

- **Academic Sources:** PubMed Central, ArXiv, and USPTO Backgrounds provide dense scientific and technical reasoning.
- **Internet & Prose:** Common Crawl, Wikipedia, and Project Gutenberg provide general world knowledge and narrative structures.
- **Dialogue & Code:** Reddit, StackExchange, and GitHub provide conversational flow, logical deduction, and structured syntax.
- **The Result:** Billions of Unicode words creating a weak but incredibly vast knowledge base across all human disciplines.



Source Category	Source Name	Illustrative Data Volume	Vast Coverage Rating
Academic Sources	Common Crawl	11.33 billion	Weak
Internet & Prose	StackExchange	32.66 billion	Weak
Dialogue & Code	GitHub	36.25 billion	Vast

Architectures: **Pretraining**

The structure of a neural network fundamentally changes how it must be pretrained and what applications it will eventually excel at.

Architectural Feature	Encoders	Decoders
Context Window	Bidirectional (Views left and right)	Unidirectional (Autoregressive)
Future Conditioning	Can condition on future words	Cannot condition on future words
Natural Pretraining	Requires custom objectives	Standard Language Modeling fits naturally
Primary Strength	Deep text understanding & Classification	Sequential text generation

Architectures: Encoders

Focusing specifically on Encoders: their unique ability to observe both left and right context simultaneously grants deep understanding, but complicates standard training.

Architectural Feature	Encoders	Decoders
Context Window	Bidirectional (Views left and right)	Unidirectional (Autoregressive)
Future Conditioning	Can condition on future words	Cannot condition on future words
Natural Pretraining	Requires custom objectives	Standard Language Modeling fits naturally
Primary Strength	Deep text understanding & Classification	Sequential text generation

Encoder Pretraining Objective

How do we train a model that already sees the answer?

The Challenge

Standard language modeling (predicting the very next word) fundamentally fails for encoders.

Because encoders view the entire sequence simultaneously and bidirectionally, asking it to predict word x_N is trivial—the model simply "looks ahead" at word x_N already present in the input, learning nothing about language semantics in the process.

The Solution: Masked LM

To force the model to learn, we must hide information.

The Masked Language Modeling (MLM) objective replaces a fraction of the input words with a special [MASK] token. The model is then penalized if it cannot accurately predict the original word based strictly on the surrounding unmasked context.

"I [MASK] to the [MASK]" → "went", "store"

BERT: Masking Strategy

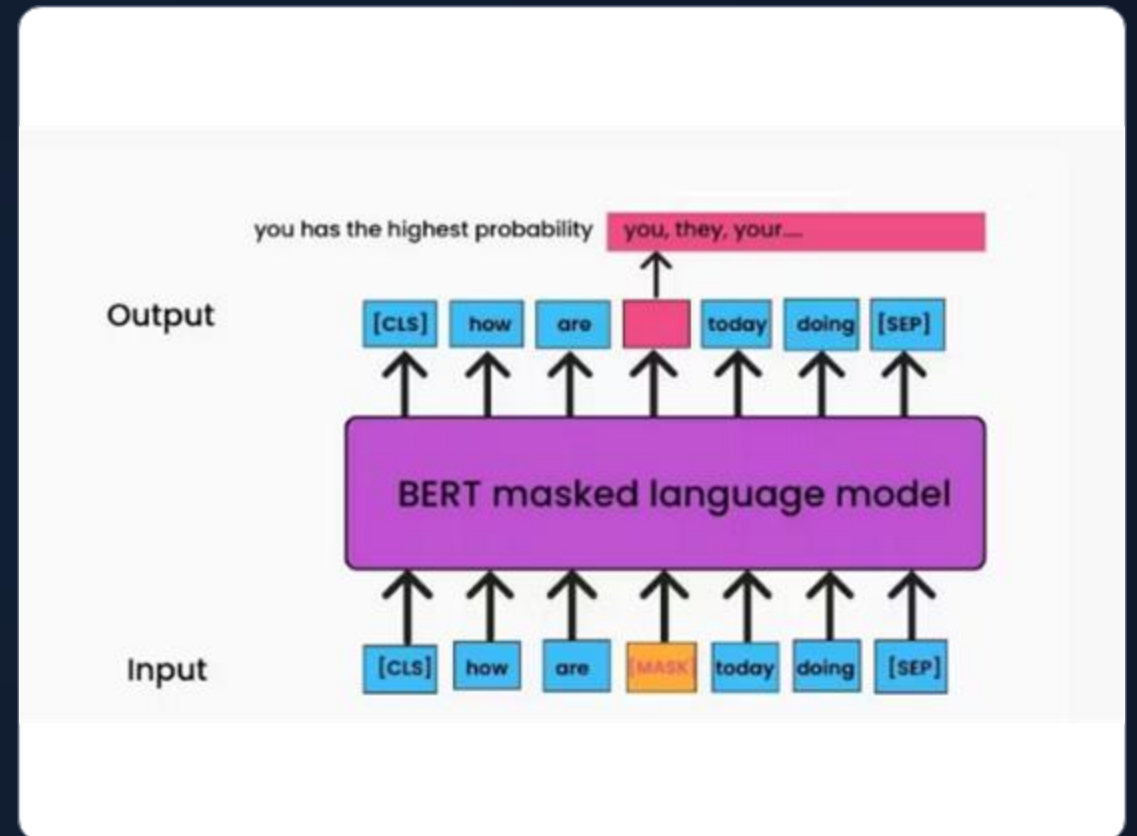
Bidirectional Encoder Representations

BERT, introduced by Devlin et al. (2018), popularized the Masked LM objective using a highly specific and robust masking strategy. For every sequence, a random 15% of subword tokens are chosen for the prediction task.

Of that selected 15%:

- **80%** are replaced with the actual [MASK] token.
- **10%** are replaced with a completely random token.
- **10%** are left entirely unchanged.

The Purpose: Because the model never sees [MASK] tokens during downstream finetuning, this mixed strategy prevents the model from getting complacent, forcing it to build strong, robust representations for every single word.



BERT: Input Representation

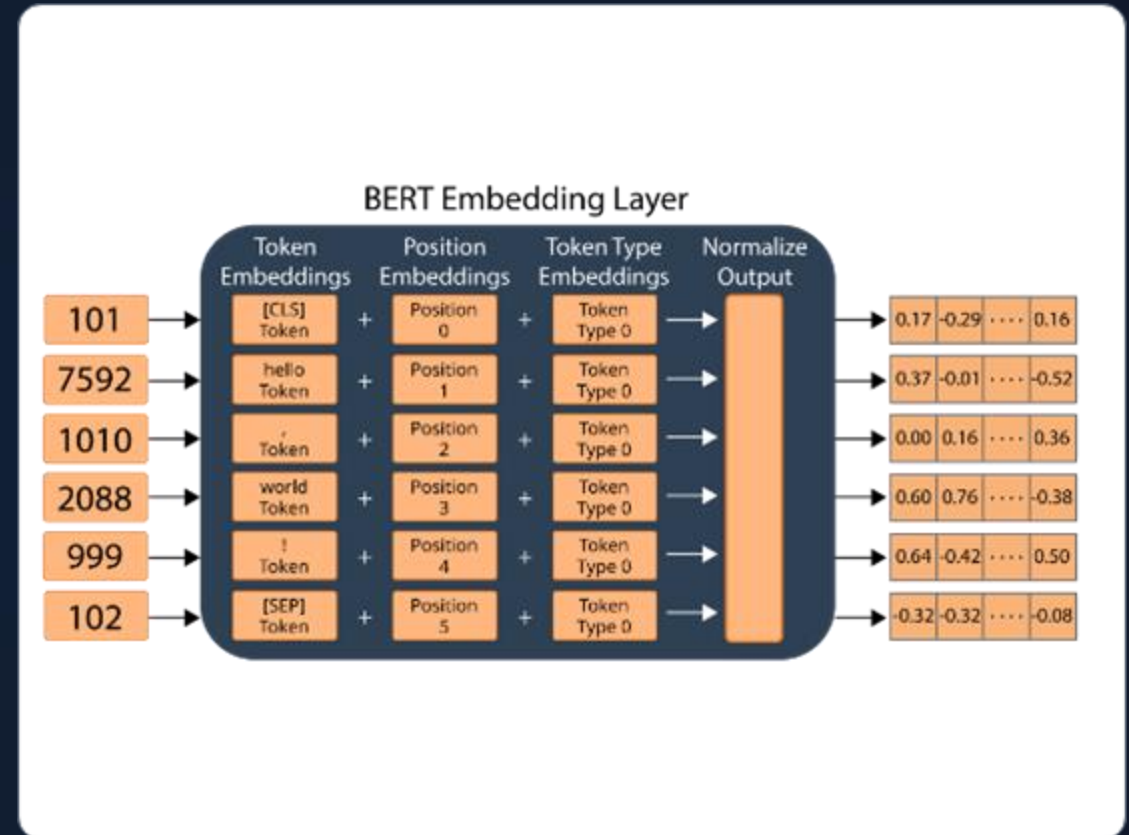
Structuring the Sequence

Before text is fed into the Transformer blocks, it must be properly formatted to give the model spatial and logical context.

The input often consists of two separate chunks (e.g., "my dog is cute" and "he likes playing"). The final embedding for any single word is calculated by summing three distinct layers:

- **Token Embeddings:** The actual subword vocabulary representation.
- **Segment Embeddings:** Indicates if the token belongs to chunk A or chunk B.
- **Position Embeddings:** Indicates the token's sequential order in the total string.

Special [CLS] (Classification) and [SEP] (Separator) tokens are heavily utilized.



BERT: Configuration Details

The original BERT release outlined two primary configurations, both requiring massive, expensive compute for pretraining, but becoming highly accessible for downstream use.

Model Configuration	Transformer Layers	Hidden Size	Total Parameters
BERT-Base	12	768	110 Million
BERT-Large	24	1024	340 Million

Training Corpora

Trained on the **BooksCorpus** (800 million words) and **English Wikipedia** (2,500 million words).

Computational Reality

Pretraining required **64 TPUs for 4 days**. Finetuning is highly practical on a **single standard GPU**.

BERT: Empirical Results

Finetuning BERT established entirely new state-of-the-art results across a broad range of natural language understanding tasks on the GLUE benchmark.

Model (Prior & BERT)	SST-2	CoLA	MRPC	QNLI	QQP
BiLSTM + ELMo + Attn	80.2	35.0	78.0	79.8	65.1
OpenAI GPT (Decoder)	91.3	45.4	82.3	88.1	70.3
BERT-Base	92.7	52.1	86.7	88.4	71.2
BERT-Large	94.9	60.5	89.3	92.7	72.1

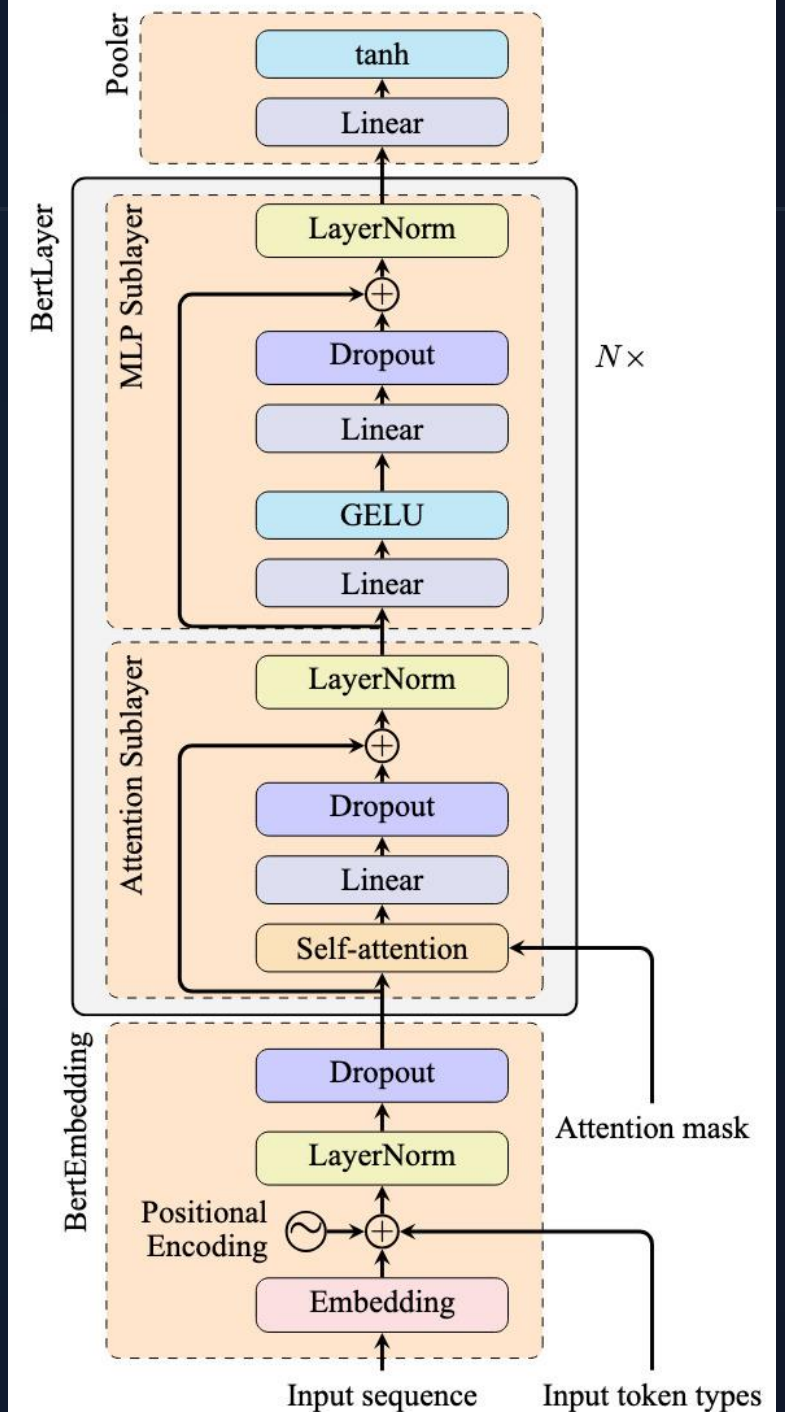
BERT-Large demonstrated significant, undeniable performance leaps across every metric compared to preceding architectures.

Encoder Limitations

Why not use them for everything?

Despite their exceptional empirical results on understanding tasks, pretrained encoders possess a critical structural flaw: **they are poorly suited for generation tasks.**

- **Bidirectional Interference:** Because BERT looks at text bidirectionally to resolve context, it fundamentally breaks the rules of generating text organically.
- **Autoregressive Mismatch:** Generating a coherent response requires an autoregressive (one-word-at-a-time) approach, looking only at the past to predict the future. Encoders attempting to sequentially fill `[MASK]` tokens generate clunky, inaccurate outputs.
- **The Decoder Shift:** If a task involves generating sequences (like writing an essay or summarizing a document), a pretrained Decoder is definitively the preferred choice.

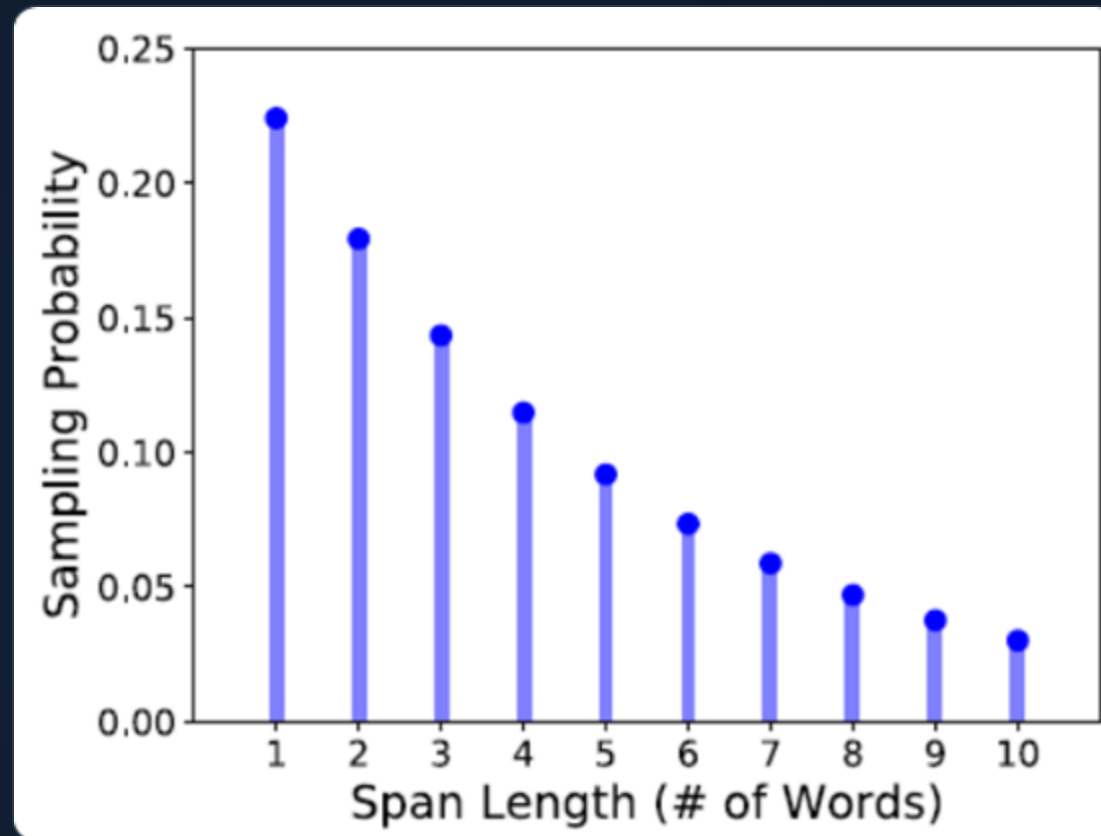


Extensions of BERT

Improving the Pretraining Formula

Following BERT's release, researchers immediately began optimizing its architecture and training objectives to achieve even higher performance.

- **RoBERTa:** Improved performance significantly by training the BERT architecture for much longer periods, on larger batches, and over more data. Crucially, it completely removed BERT's "next sentence prediction" objective, proving it was unnecessary.
- **SpanBERT:** An evolution of the masking strategy. Instead of masking isolated random subwords across the text, SpanBERT replaces contiguous blocks (spans) of words with masks.
- **The Advantage:** Predicting an entire continuous span forces the model to rely more heavily on long-range context, creating a much harder and more useful pretraining task than simply guessing a single random missing word.



Extensions of BERT (Scaling)

A massive takeaway from the RoBERTa paper was the realization that simply **increasing compute and data** drastically improves pretraining outcomes, without altering the underlying Transformer encoder architecture.

Model Setup	Dataset Size	Training Steps	SQuAD (v1.1)	MNLI (m/mm)	SST-2
BERT-Large (Original)	16 GB	100K	90.9	86.6 / 86.3	93.2
RoBERTa (BERT Setup)	16 GB	100K	93.6	88.5 / 88.3	94.8
RoBERTa (More Data)	160 GB	100K	94.0	89.3 / 89.0	95.3
RoBERTa (More Steps)	160 GB	500K	94.6	90.2 / 90.2	96.4

By expanding from 16GB to 160GB of text and pushing training steps to 500K, RoBERTa systematically broke past original BERT-large limitations.

Architectures: Decoders Focus

Mastering Sequence Generation

Returning to our architectural comparison, we now shift our focus entirely to **Decoders**. While encoders excel at bidirectional understanding, decoders are strictly autoregressive. They lack the ability to condition on future words, observing only the past context to predict the next word. Crucially, because this aligns perfectly with how human text is generated sequentially, **all of the largest pretrained models currently in use are based on the Decoder framework.**

Architectural Feature	Encoders	Decoders
Context Window	Bidirectional (Views left and right)	Unidirectional (Autoregressive)
Future Conditioning	Can condition on future words	Cannot condition on future words
Natural Pretraining	Requires custom objectives	Standard Language Modeling fits naturally
Primary Strength	Deep text understanding & Classification	Sequential text generation

Pretraining Decoders (Generation)

The Natural Use Case: Text Generators

$$p_{\theta} (w_t \mid w_1 : t - 1)$$

Pretrained decoders function inherently as powerful text generators. By finetuning the conditional probability distribution shown above—where the probability of the current word is mathematically conditioned on all previous words—these models excel at sequence-to-sequence tasks.

This autoregressive capability is exceptionally helpful for dynamic applications. In **dialogue generation**, the model generates responses based on the context of dialogue history. In **summarization**, it generates concise takeaways based on the full document context.

Generative Pretrained Transformer (GPT)

The First Major Decoder Success

The original GPT model, introduced in 2018 by Radford et al. at OpenAI, marked a massive success in pretraining a Transformer decoder architecture.

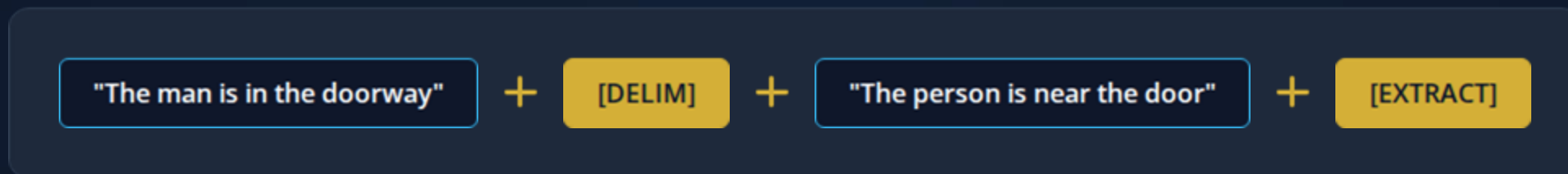
- **Architecture Details:** The model featured 12 transformer blocks (layers), 117 million parameters, and 768-dimensional hidden states.
- **Vocabulary:** It utilized a highly efficient byte-pair encoding (BPE) vocabulary consisting of 40,000 merges.
- **Training Data:** The model was trained extensively on the BooksCorpus. This dataset was specifically chosen to help the model learn long-distance dependencies and narrative coherence.
- **Fun Trivia:** Despite defining an entire generation of AI, the famous acronym "GPT" did not actually appear anywhere in the original paper's text.



GPT: Input Formatting

Because decoders naturally generate text sequentially, applying them to classification tasks requires clever input formatting. This is how raw text inputs are structured so a decoder can process them for finetuning on Natural Language Inference (NLI) tasks.

Textual Entailment Sequence



Linear Classifier Prediction

An example premise and hypothesis are formatted sequentially into a single string. The texts are separated by a delimiter token and end with an [EXTRACT] token. The linear classifier uses the final state of this extract token to make its prediction.

GPT: Initial Results

A comprehensive look at the original GPT's performance on various Natural Language Inference (NLI) datasets. The finetuned Transformer language model achieved highly competitive or state-of-the-art results against specialized baseline models.

Dataset	ESIM + ELMo (Baseline)	GenSen (Baseline)	GPT (Finetuned)
MNLI (Matched)	80.6	71.4	82.1
MNLI (Mismatched)	80.1	71.3	81.4
SNLI	88.6	87.3	89.9
SciTail	83.3	80.4	88.3
QNLI	81.1	-	88.1
RTE	56.8	59.3	56.0

Increasingly Convincing Generations (GPT-2)

Scaling to 1.5 Billion Parameters

OpenAI's subsequent release, GPT-2, highlighted a massive leap in raw generation quality simply by scaling up the model size and training data.

- **The Famous Prompt:** A human provided context about scientists discovering a herd of English-speaking unicorns living in a remote, previously unexplored valley in the Andes Mountains.
- **The Output:** GPT-2 generated a remarkably coherent narrative detailing the discovery.
- **Emergent Creativity:** The model invented logical scientific names like "Ovid's Unicorn" and maintained a highly convincing, grounded journalistic tone throughout the generated article, proving large decoders could maintain long-term narrative consistency.



GPT-3 and In-Context Learning

A Paradigm Shift

GPT-3 introduced a fundamental shift in how humans interact with artificial intelligence, moving away from the "finetuning" phase entirely and toward "**in-context learning**".

Researchers discovered that very large language models are capable of performing complex tasks *without any gradient updates or parameter changes*. They accomplish this simply by recognizing patterns based on the examples provided within their text context window.

The Scale Difference

This emergent capability was unlocked through unprecedented scale.

T5 (Previous Largest)

11 Billion Parameters

GPT-3 (2020)

175 Billion Parameters

In-Context Learning: Translation Example

By providing a prompt prefix that specifies a task, the model learns the objective entirely within the context window. Observing the conditional distribution established in the prompt, the model successfully generates the correct output, often matching the performance of specifically trained translation models.

Translate English to French:

sea otter => loutre de mer

peppermint => menthe poivrée

plush girafe => girafe en peluche

mint => menthe

otter => loutre ■

Versatility of In-Context Learning

The same underlying model can perform entirely different tasks based solely on the structural patterns provided in the prompt, showcasing incredible zero-shot and few-shot versatility.



Mathematical Addition

5 + 8 = 13

7 + 2 = 9

4 + 4 = 8

The model acts as a calculator, recognizing arithmetic sequences simply by viewing previous solved examples in the text prompt.



Word Unscrambling

gaot => goat

pplea => apple

The model infers the task is an anagram solver. Without specific tuning, it corrects the misspelled target word accurately.



Language Translation

cheese =>

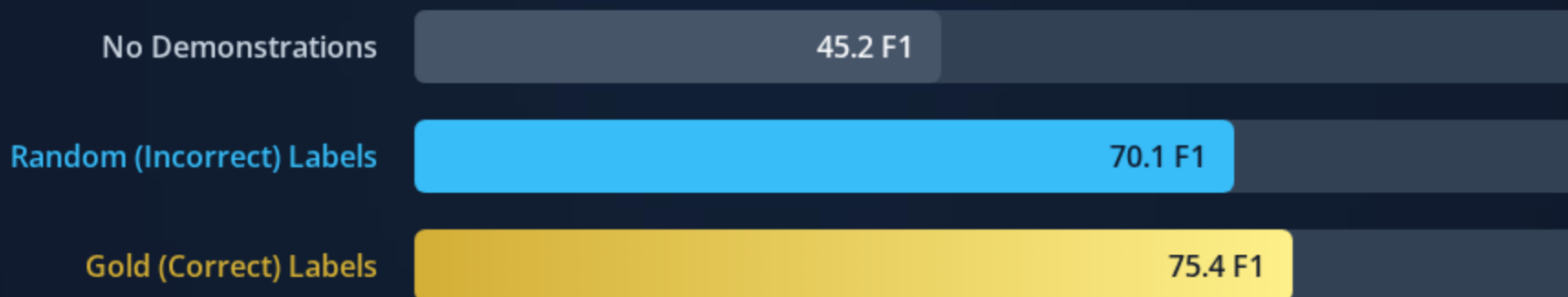
fromage

water => eau

The model rapidly shifts its internal weights to map English tokens to French tokens, mocking dedicated translation architectures.

How do LLMs generalize? (Prompt Labels)

Are large models truly "learning" the task from the examples provided? To test this, researchers measured the Macro-F1 performance across three distinct prompt conditions to understand in-context generalization.



Surprising Finding: Models perform extraordinarily well on tasks even when the in-context examples have completely randomized, incorrect labels. The gap between Random and Gold labels is remarkably small compared to providing no demonstrations at all.

How do LLMs generalize? (Mechanisms)

Complex Generalization Behavior

Testing model accuracy across various label conditions (Abstract, Default, Flipped) reveals that Large Language Model (LLM) behavior is highly complex and not uniform.

- **Abstract Labels:** Using nonsensical markers still yields strong performance, proving the model leverages the input distribution.
- **Flipped Labels:** Forcing the model to output "Negative" for positive text heavily damages performance, showing it relies on prior pretraining knowledge.

The Core Mechanism

The key takeaway is that in massive models, the mechanism underlying in-context learning is a dual mixture of capabilities:

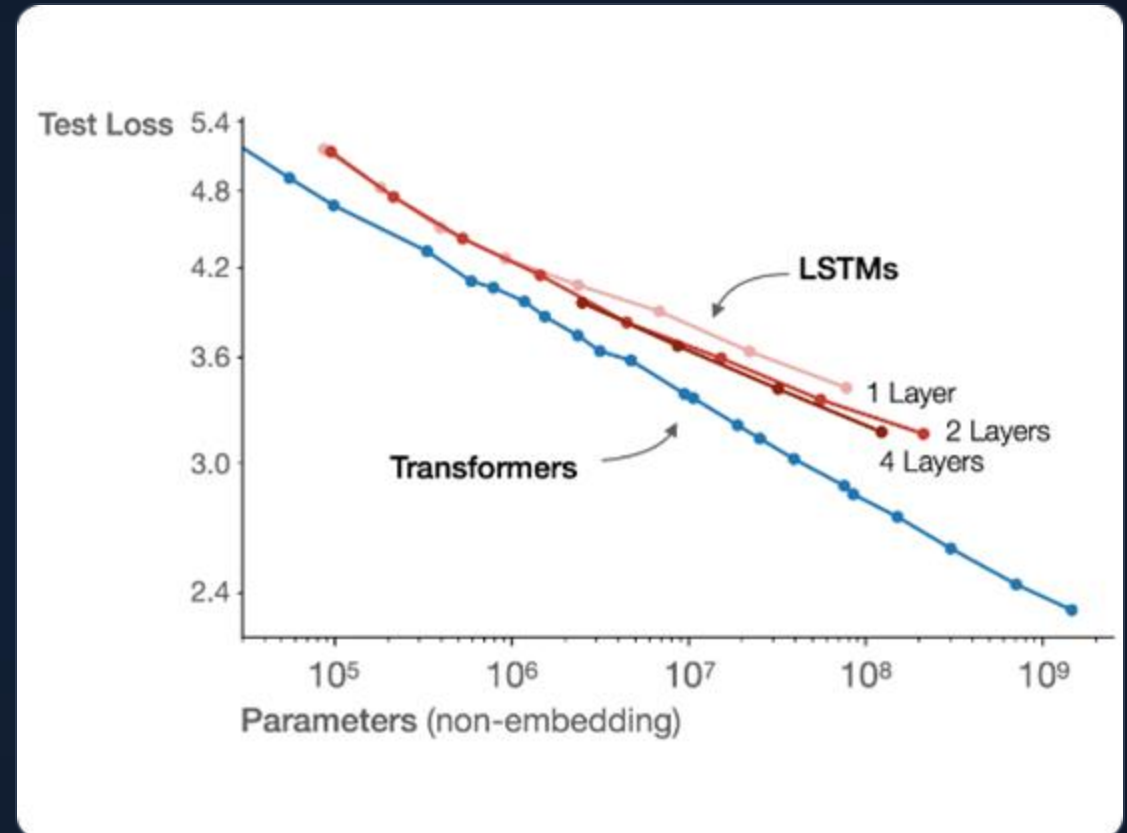
- **Format Inference:** The model infers the desired formatting and output space of the task simply from the structure of the prompt.
- **Novel Learning:** Concurrently, it actually learns novel information from the provided examples to augment its pre-existing knowledge.

Why scale? Empirical Scaling Laws

Predictable Performance Gains

The concept of empirical "scaling laws" fundamentally altered deep learning strategy. The laws state that predictably scaling up resources leads to reliable, continuous gains in model performance (measured via test loss / perplexity).

- **Compute:** When plotted on a log-log scale, increasing the total compute budget results in a straight, downward trend in test loss.
- **Dataset Size:** Continuously expanding the size of the training corpus directly improves the model's generalization capabilities.
- **Parameter Count:** Increasing the model's internal parameter count systematically yields a more capable architecture.
- **The Guarantee:** These power laws provide a mathematical guarantee that investing more resources will yield a proportionally better model.



Scaling Laws for **Architecture Decisions**

Predictability Before Computation

Predictable performance scaling applies to more than just raw dataset size; it heavily influences architectural configurations as well.

- **Layer Scaling:** Graphs demonstrate predictable test loss improvements when scaling the number of model layers (depth).
- **Architecture Comparison:** These scaling laws allow researchers to directly compare completely different architectures. For instance, log-log plots clearly highlight when a Transformer will begin outperforming an LSTM given sufficient scale.
- **Strategic Planning:** This predictability helps AI engineers make intelligent, data-driven decisions about model depth, width, and dataset ratios *before* spending millions of dollars on compute.



Scaling Efficiency: Using Compute Optimally

While scale guarantees better performance, *efficiency* dictates how to balance that scale. The optimal ratio is balancing the number of parameters against the number of training tokens.

Model	Parameter Count	Training Tokens	Ratio (Tokens per Param)	Result Status
OpenAI GPT-3	175 Billion	300 Billion	~ 1.7x	Severely Under-trained (Suboptimal)
DeepMind Gopher	280 Billion	300 Billion	~ 1.1x	Under-trained
DeepMind Chinchilla	70 Billion	1.4 Trillion	20x	Compute-Optimal (Outperformed GPT-3)

DeepMind's "Chinchilla" model proved that GPT-3 did not strike the right balance. Despite having only 70 billion parameters, Chinchilla outperformed much larger models because it was compute-optimally trained on a significantly larger corpus of 1.4 Trillion tokens.